Git + GitHub 核心概念串讲
从零到一攻略

用动画讲解 + 实战演示的方式，把提交、分支、分区、远端协作、Rebase 等核心概念一次串清。

15 个章节　零基础友好　可视化图解

01 引言

为什么在 AI 时代，Git 反而变成了「必修基本功」？

为什么必须学 Git

- **代码是资产，版本控制是安全网**——删错一行、改崩一个功能，随时能回到上一秒的完整状态。
- **AI 生成的代码更需要版本管理**——AI 一次可能给你改十几个文件，你必须有办法「一键撤回」。
- **团队协作的公共语言**——全世界几乎所有软件公司都用 Git，不会它就没法参与真实项目。
- **GitHub 是程序员的「简历 + 作品集」**——开源贡献、个人项目都沉淀在这里。

本课的学习路径

第一步：概念

搞清楚 Git 是什么、GitHub 是什么、Commit 是什么，先建立心智模型。

第二步：本地实战

Init → Commit → Branch → Merge，把仓库在自己电脑上先玩转。

第三步：连上远程

Push / Pull / Clone，把代码备份到 GitHub，开始多人协作。

第四步：进阶操作

Cherry Pick、Stash、Rebase，处理真实项目里的复杂场景。

学习建议

Git 是「动手型」知识，建议边看边在终端里敲。

02 Git 与 GitHub 概念

最容易搞混的一对概念：一个是**工具**，一个是**网站**。

Git

版本控制系统，一个装在你自己电脑上的**软件**。
负责记录每一次改动、管理分支、合并代码。
不联网也能用。

GitHub

全球最大的**代码托管网站**，一个**平台**。
用 Git 把代码推上去备份，别人可以查看、提建议、一起改。
必须联网。

一句话类比

Git = 你家里的日记本 + 时光机（记录你每天写的每一版）；
GitHub = 云端的图书馆 + 协作广场（把日记本上传，还能让别人一起写）。

核心概念：Commit（提交）

整个 Git 世界都围绕 **Commit** 展开。一次 Commit 就是**一次完整的快照**—— 它把你项目在那一刻的所有文件状态完整保存下来，并附上一个说明、一个时间戳、一个唯一的哈希值。



版本控制解决了什么问题

没有 Git	有 Git
项目_v1.zip、项目_v2_最终版.zip、项目_真的最终版.zip…	一条清晰的提交历史线
改崩了只能靠记忆手动还原	`git revert` 一键撤销某次改动
多人改同一份文件只能互相覆盖	分支 + 合并，各自开发互不干扰
不知道这行代码是谁写的、为什么写	`git blame` 精确到人、精确到提交

03 实战准备

三步搞定环境：装 Git → 配身份 → 注册 GitHub。

第 1 步：安装 Git

- 官网：`git-scm.com`，下载对应系统安装包，一路「下一步」即可。
- Windows 用户安装时会自带 **Git Bash**，后续命令都在它里面敲。
- Mac 用户可直接在终端输入 `git`，系统会提示安装命令行工具。

第 2 步：验证安装

```
git --version
# 输出类似：git version 2.43.0
```

第 3 步：配置身份（非常重要）

Git 需要知道「每次提交是谁做的」，这个信息会永久写进每一条 Commit 记录里。

```
# 全局配置用户名和邮箱（只需做一次）
git config --global user.name "你的名字"
git config --global user.email "你的邮箱@example.com"

# 查看当前配置
git config --global --list
```

> ⚠ **易错点**
>
> 邮箱建议用你注册 GitHub 时用的那个邮箱，这样 GitHub 才能把提交正确归属到你的账号（显示头像和贡献绿格子）。

第 4 步：注册 GitHub

1. 打开 `github.com`，点击 **Sign up**。
2. 填写用户名（会成为你的个人主页地址：`github.com/你的用户名`）、邮箱、密码。
3. 完成邮箱验证。
4. （推荐）上传头像、填写个人简介，让它看起来像个正经作品集。

> 💡 **可选但推荐**
>
> 配置 SSH 密钥可以免去每次 push 都输密码的麻烦。命令：`ssh-keygen -t ed25519 -C "邮箱"`，然后把 `~/.ssh/id_ed25519.pub` 的内容粘贴到 GitHub 的 Settings → SSH and GPG keys。

04 Init 初始化

把一个普通文件夹，变成「Git 能管理的仓库」。

什么是仓库（Repository）

仓库 = 被 Git 纳管的目录。初始化之后，文件夹里会多出一个隐藏的 `.git` 目录， 里面存着**全部版本历史、分支、配置**。可以说：**有 .git 目录的文件夹才叫仓库。**

```
# 1. 进入你的项目文件夹
cd ~/my-project

# 2. 初始化仓库
git init

# 输出：Initialized empty Git repository in .../my-project/.git/
```

初始化之后长什么样

```
my-project/
├── .git/ ← Git 的「大脑」，所有历史都在这里
│   ├── objects/   所有文件快照
│   ├── refs/      分支指针
│   └── config     仓库配置
├── index.html    ← 你的项目文件
└── README.md
```

绝对不要做的事

不要手动删除、修改、移动 `.git` 目录里的任何文件——那等于直接破坏版本历史，基本无法恢复。
删掉整个 `.git` 文件夹 = 这个项目瞬间「失忆」，所有提交记录全没。

查看状态：你最常用的命令

```
git status

# On branch main
# No commits yet
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#         index.html
```

`git status` 是你随时可以敲的「GPS」——它告诉你：现在在哪个分支、哪些文件改了、哪些已经暂存。

05 Commit 提交

Git 最核心的动作：把改动「定格」成一个版本。

两步走：add → commit

```
# 1. 把文件加入暂存区（告诉 Git：这些改动我要提交）
git add index.html
git add .          # 添加当前目录所有改动

# 2. 提交，并写上一句说明
git commit -m "feat: 新增首页导航栏"
```



一次提交 = 把暂存区的内容打包成一个永久快照

Commit Message 规范

写得好，半年后你自己还能看懂。业界通用的 **Conventional Commits** 约定：

前缀	含义	示例
`feat`	新功能	`feat: 新增用户登录功能`
`fix`	修复 bug	`fix: 修复首页按钮点击无响应`
`docs`	文档修改	`docs: 更新 README 安装说明`
`style`	格式调整（不影响逻辑）	`style: 统一缩进为 2 空格`
`refactor`	重构	`refactor: 拆分用户服务类`
`chore`	杂项/构建	`chore: 升级依赖版本`

> ⚠ **常见错误写法**
>
> ✗ `"改了一下"` ✗ `"update"` ✗ `"123"` ✗ `"阿巴阿巴"`
>
> 好的提交信息 = 别人看这一行就知道这次改了什么、为什么改。

查看历史与差异

```
git log --oneline       # 一行一条的精简历史
git log --oneline --graph # 带分支图的历史
git diff                # 工作区 vs 暂存区 的差异
git diff --staged       # 暂存区 vs 上次提交 的差异
git show <commit-id>    # 查看某次提交的详情
```

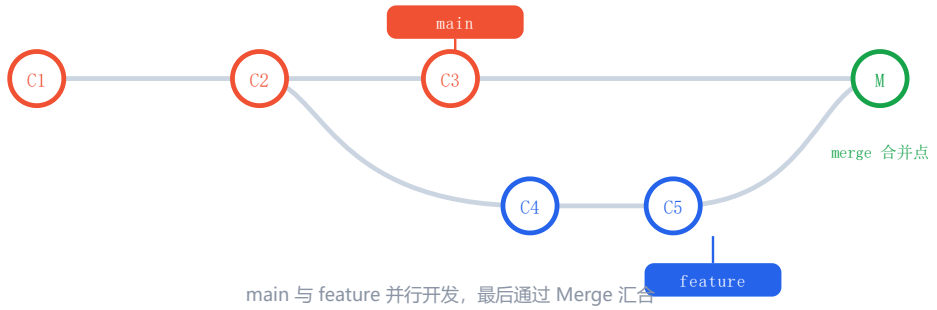
> 💡 **小技巧**
>
> `git commit -am "msg"` 可以跳过 `add` 直接提交**已被 Git 跟踪过**的文件的修改。但新增的文件（Untracked）仍然必须先 `add`。

06 Branch 分支

Git 最强大的能力：让你同时推进多条开发线。

分支的本质：它只是一个「指针」

很多人以为创建分支会把代码复制一份——**完全不是。**分支在 Git 里只是一个 **指向某次 Commit 的轻量指针**，创建成本几乎为零（几十字节）。



main 与 feature 并行开发，最后通过 Merge 汇合

分支常用命令

```
# 查看所有分支（* 号表示当前所在分支）
git branch

# 创建分支（只创建，不切换）
git branch feature-login

# 创建并立即切换（最常用）
git checkout -b feature-login
git switch -c feature-login      # 新版写法

# 切换分支
git checkout main
git switch main

# 把 feature-login 合并进当前分支
git merge feature-login

# 删除已合并的分支
git branch -d feature-login
git branch -D feature-login   # 强制删除未合并的
```

HEAD 是什么

`HEAD` 是 Git 里的「**你正在哪儿**」标记。它通常指向某个分支，而那个分支再指向某个 Commit。

⚠ **分离头指针（Detached HEAD）**

如果你用 `git checkout <commit-id>` 直接切到某次历史提交，HEAD 就不指向任何分支了。 此时你的新提交**不属于任何分支**，一旦切走就会变成「游离提交」，很容易丢失。
正确姿势：想在那个位置改代码，先 `git switch -c new-branch` 建个分支再动手。

✅ **分支使用惯例**

`main`（或 `master`）永远是**可运行、可上线**的稳定版本； 每个新功能 / 每个 bug 修复都从 main 拉一个独立分支；做完再合并回去。这样 main 永远不会被搞坏。

07 WorkTree 工作树

同一个仓库，同时在多个文件夹里工作。

它解决什么痛点

你正在 `feature-A` 分支写代码写到一半，突然线上出 bug 需要立刻修。 传统做法是 `git stash` 存一下再切分支，切回来还要 stash pop，很折腾。

Worktree 的解法：给另一个分支单独开一个文件夹，两个文件夹共享同一个 Git 历史， 你在 A 文件夹写功能、在 B 文件夹修 bug，互不干扰。

```
# 在当前仓库旁边新建一个工作树文件夹，检出 hotfix 分支
git worktree add ../my-project-hotfix hotfix

# 查看当前有哪些工作树
git worktree list

# 用完删除工作树（不影响分支本身）
git worktree remove ../my-project-hotfix
```

```
~/projects/
├── my-project/          ← 主工作树，当前在 feature-A 分支
│   ├── .git/            ← 真正的 Git 数据（只有这里有一份）
│   └── src/...
│
└── my-project-hotfix/   ← 第二工作树，当前在 hotfix 分支
    ├── .git             （只是个指向主仓库的指针文件）
    └── src/...
```

适用场景

① 紧急修复线上 bug；② 同时对比两个分支的代码效果；③ 一个分支跑测试、一个分支继续写代码；④ 构建/打包时不想打断当前开发。

注意

同一个分支**不能**同时被两个工作树检出，会报错。这是有意设计的——防止两处同时改同一分支造成混乱。

08 Conflict 冲突

Git 不是万能的，有些事必须人来决定。

冲突是怎么产生的

当两个分支**修改了同一个文件的同一行区域**，Git 无法判断该保留哪一个，于是停下来，把决定权交给你。



merge 时两个分支改动同一行 → 触发 CONFLICT，需要人工裁决

冲突长什么样

打开冲突文件，会看到 Git 插入的特殊标记：

```
<<<<<<< HEAD
let color = "blue";       // 当前分支（ours）的内容
=======
let color = "red";        // 要合并进来的分支（theirs）
>>>>>>> feature
```

解决冲突四步法

1. **看**：`git status` 会列出所有 *both modified* 的文件。
2. **改**：打开文件，手动决定最终保留哪段代码（可以两个都留，也可以都不留），然后**删除** `<<<<<<<`、`=======`、`>>>>>>>` **这三行标记。**
3. **标记已解决**：`git add <文件名>`。
4. **完成合并**：`git commit`（Git 会自动生成一条 merge commit 信息）。

```
git status                  # 找出冲突文件
# ...手动编辑文件，删掉冲突标记...
git add conflicted-file.js
git commit

# 如果搞砸了，想放弃这次合并回到合并前
git merge --abort
```

减少冲突的习惯

① 分支生命周期尽量短，别让一个分支活一个月；② 频繁从 main 拉取最新代码合并进来；③ 团队约定同一个文件的负责人；④ 编辑器插件（VS Code 有内置的三栏冲突解决界面）能大幅提速。

09 分区模型（四区）

整门课最核心的心智模型：搞懂这四个区，Git 就懂了一半。

四区全景图



← 反向流动：git restore / git reset / git pull

区域	位置	含义	进入命令	回退命令
工作区	你电脑上的文件夹	你正在编辑的代码，肉眼可见	编辑文件	`git restore <file>`
暂存区	`.git/index`	提交前的「购物车」，可以挑选要提交的内容	`git add`	`git restore --staged <file>`
本地仓库	`.git/objects`	所有已提交的历史版本，永久保存	`git commit`	`git reset` / `git revert`
远程仓库	GitHub 服务器	团队共享的中央仓库	`git push`	`git pull`（拉回来）

📌 **为什么需要「暂存区」这个中间层?**

因为它让你可以**挑着提交**。你改了 5 个文件，但只想提交其中 2 个相关的改动—— 先 `add` 那 2 个，然后 `commit`，剩下的留在工作区继续改。 如果没有暂存区，你只能「全提交」或「全不提交」。

三种「后悔药」

不同场景要用不同的药，用错会出大问题：

① Discard 放弃修改

场景：改动还没 commit，想直接扔掉。

`git restore <file>`

⚠ 丢弃后无法找回

② Reset 回退

场景：想把自己本地分支的历史「抹掉」几次提交。

`git reset --hard HEAD~1`

⚠ 会改写历史，**多人分支禁用**

③ Revert 反向提交

场景：撤销某次已推送的提交，但保留历史。

`git revert <commit-id>`

✅ 多人协作首选

```
# ---- Reset 的三种模式 ----
git reset --soft HEAD~1   # 撤销 commit，改动保留在暂存区
git reset --mixed HEAD~1  # 撤销 commit 和 add，改动保留在工作区（默认）
git reset --hard HEAD~1   # 彻底撤销 commit + add + 工作区改动（危险）

# ---- Revert：生成一条「反向提交」来抵消目标提交 ----
git revert <commit-id>
git revert --no-commit <commit-id>  # 先不自动提交，手动调整
```

⛔ **黄金铁律**

多人协作的公共分支（main / develop）上，绝对禁止使用 `git reset`。

因为你改写的是「历史」，而别人电脑上的历史还是旧的。你一 push（还得 `-f` 强推），所有人的提交记录就和服务器对不上了，会导致别人的工作丢失、后续无法正常合并。

线上出问题要撤销，永远用 `git revert`。

命令与区域的对应关系（背下来）

命令	方向	作用
`git add`	工作区 → 暂存区	挑选要提交的改动

命令	方向	作用
`git restore --staged`	暂存区 → 工作区	把改动移出暂存区
`git commit`	暂存区 → 本地仓库	生成一个版本快照
`git reset`	本地仓库 → 暂存区/工作区	回退提交（改写历史）
`git push`	本地仓库 → 远程仓库	上传提交
`git pull`	远程仓库 → 本地仓库+工作区	下载并合并（= fetch + merge）
`git fetch`	远程仓库 → 本地仓库	只下载，不合并

10 远端仓库实战

把本地代码推上云端，从此不再怕硬盘坏。

两种起点：从零开始 or 克隆别人的

场景 A：本地已有项目，要推到 GitHub

```
# 1. 在 GitHub 网页上点 New repository，创建一个空仓库（不要勾选 README）
# 2. 在本地项目里关联远程地址，并给它起名叫 origin
git remote add origin https://github.com/你的用户名/仓库名.git

# 3. 确认分支名叫 main（GitHub 默认）
git branch -M main

# 4. 首次推送，-u 表示把本地 main 和远程 origin/main 绑定
git push -u origin main
```

场景 B：远程已有项目，要拉到本地

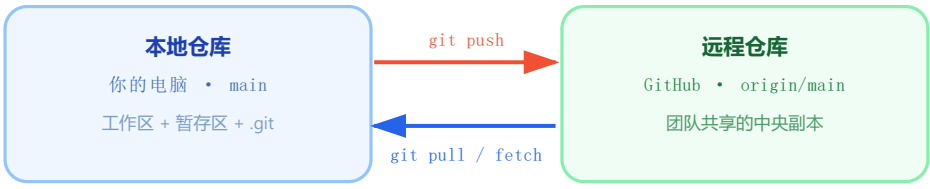
```
git clone https://github.com/用户名/仓库名.git

# 克隆会自动完成三件事：
# 1. 下载全部代码和历史
# 2. 创建 origin 远程别名
# 3. 检出默认分支到工作区
```

origin 是什么

`origin` 只是**远程仓库地址的别名**，不是关键字。你也可以叫 `upstream`、`github`，但业界约定俗成都用 `origin` 表示「我 fork/克隆来源的那个仓库」。

```
git remote -v            # 查看已配置的远程地址
git remote set-url origin <新地址>   # 修改远程地址
git remote remove origin           # 删除远程关联
```



⚠ **push 被拒绝怎么办**

报错 `! [rejected] ... fetch first`，说明远程有你本地没有的提交。**正确做法**：先 `git pull` 把远程变化合并进来，解决可能的冲突，再 `git push`。
错误做法：直接 `git push -f` 强推——这会覆盖掉别人的提交。

💡 **实用命令**

`git push origin --delete 分支名` 删除远程分支；
`git remote show origin` 查看远程仓库的详细信息。

11 GitHub 入门

不只是一个网盘，而是一个完整的协作生态。

创建仓库时的几个关键选项

选项	作用	建议
Repository name	仓库名，会出现在 URL 里	用英文小写 + 连字符，如 `my-portfolio`
Public / Private	公开可见 / 仅自己（及协作者）可见	作品集用 Public，公司项目用 Private
Add a README file	自动生成项目说明文件	✅ 强烈建议勾选
Add .gitignore	选择要忽略的文件模板	✅ 按语言选（Node / Python 等）
Choose a license	开源协议	开源项目建议 MIT

README.md 是项目的门面

别人点进你的仓库，第一眼看到的就是 README。一个好的 README 应该包含：

- **项目简介**——一句话说清这是干什么的
- **截图 / 效果图**——视觉冲击力最强
- **安装与运行方式**——让别人能跑起来
- **技术栈**——用了什么框架、什么工具
- **License**——授权说明

README 使用 **Markdown** 语法编写，GitHub 会自动渲染成漂亮的网页。

.gitignore：不要提交不该提交的东西

有些文件**永远不该进版本库**：依赖包、编译产物、密钥文件、本地配置。

```
# .gitignore 示例（Node 项目）
node_modules/
dist/
.env
.DS_Store
*.log
```

> ⛔ **安全红线**
>
> 绝对不要把 `.env`、API Key、数据库密码、私钥文件提交到仓库——一旦推上 GitHub，**哪怕之后删掉，历史记录里依然永久存在**，爬虫几秒钟就能扫到。正确做法是用 `.gitignore` 提前排除，或者用 GitHub Secrets 管理。

必须认识的 GitHub 功能

Star ⭐

收藏 / 点赞。Star 数是开源项目影响力的直接体现。

Fork 🍴

把别人的仓库完整复制一份到你自己的账号下，你可以随意改，改完可以发起 PR 回馈原项目。

Pull Request

「我改好了，请合并」的正式请求。可以在线逐行 review、讨论、批准。GitHub 协作的核心机制。

Issues

问题追踪 / 任务清单。报 bug、提需求、讨论方案都在这里，可打标签、指派负责人。

其他实用入口

- **Actions**：CI/CD 自动化，代码一推就自动跑测试、自动部署。

- **Pages**：免费把仓库变成一个静态网站（个人主页、项目文档）。
- **Releases**：发布带版本号的软件包。
- **Settings → Collaborators**：邀请别人直接往你的仓库推代码。
- **Settings → Branches**：设置分支保护规则，防止有人直接往 main 推。

12 多人协作

从「一个人写」升级到「一群人写同一份代码」。

两种协作模型

模型一：直接推送（小团队 / 内部项目）

所有人在同一个仓库上工作，各自拉分支，做完合并回 main 再 push。 需要仓库管理员在 Settings → Collaborators 里把队友加进来。

模型二：Fork + Pull Request（开源项目 / 外部贡献）

你没有原仓库的写权限，所以先 **Fork** 一份到自己账号下，在自己那份上改， 改完向原仓库发起 **Pull Request**，由维护者审核决定是否合并。



标准协作流程（每天都会用）

```
# 1. 开始工作前，先拉取最新代码（非常重要！）
git switch main
git pull

# 2. 从最新 main 拉一个功能分支
git switch -c feature/user-profile

# 3. 写代码，多次提交
git add .
git commit -m "feat: 新增用户资料页"

# 4. 推送分支到远程
git push -u origin feature/user-profile

# 5. 在 GitHub 上发起 Pull Request，等待 review

# 6. 合并后，清理本地分支
git switch main
git pull
git branch -d feature/user-profile
```

协作中出现冲突怎么处理

1. `git pull` 之后提示冲突
2. 打开冲突文件，手动编辑解决（参考第 8 章）
3. `git add` 标记已解决
4. `git commit` 完成合并
5. `git push`

> **高频救命操作**
>
> 如果 `git pull` 产生了一个丑陋的 merge commit，可以用 `git pull --rebase` 让本地提交「接」在远程最新提交之后，历史会是一条干净的直线。 （原理见第 15 章 Rebase）

✅ **团队协作的黄金习惯**

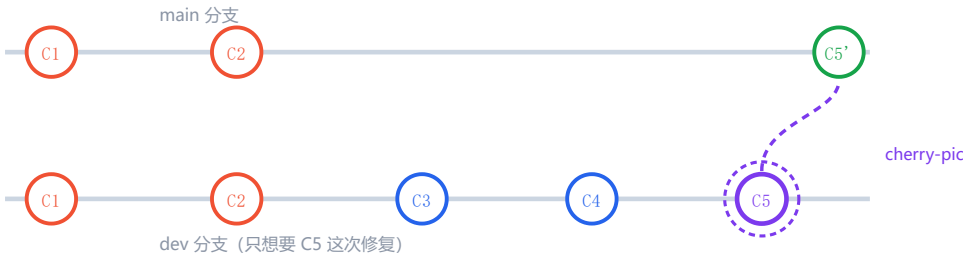
① **每天开工第一件事是 pull**；
② **一次提交只做一件事**，别把三个功能混在一次 commit 里；
③ **分支命名有意义**：`feature/xxx`、`fix/xxx`、`hotfix/xxx`；
④ **合并前先自查 diff**，别把调试代码、注释掉的代码推上去；
⑤ **main 分支加保护**，禁止直接 push，必须走 PR + review。

13 Cherry Pick

「摘樱桃」——只挑我想要的那几个提交。

它解决什么问题

你有一个开发分支 `dev`，上面有 5 个提交。现在只想把其中**那个修复 bug 的提交**单独拿到 `main` 上，其他提交暂时不要。这时候 `merge` 会把全部内容合进来，不行；`cherry-pick` 正好对症。



命令用法

```
# 1. 切换到目标分支
git switch main

# 2. 先找到你想摘的那个提交的 hash
git log --oneline dev
# a1b2c3d fix: 修复登录超时问题    ← 就是它

# 3. 摘过来
git cherry-pick a1b2c3d

# 一次摘多个（按顺序）
git cherry-pick a1b2c3d e4f5g6h

# 摘一段连续区间（左开右闭）
git cherry-pick a1b2c3d..e4f5g6h

# 只摘改动到工作区，不自动提交
git cherry-pick -n a1b2c3d
```

⚠️ **注意事项**

① Cherry-pick 会生成一个**新的 commit**（hash 不同），所以原提交和新提交是两条独立记录；
② 如果摘过来的提交依赖前面几次提交的代码，可能**产生冲突**，需要手动解决；
③ 大量使用 cherry-pick 说明分支策略可能有问题，长期来看应该优化分支模型；
④ 中途出错想放弃：`git cherry-pick --abort`。

💡 **典型使用场景**

① 把 hotfix 分支的紧急修复同步到多个版本分支；② 不小心在错误的分支上提交了，摘到正确分支去；③ 从别人的分支里「借」一个现成的功能实现。

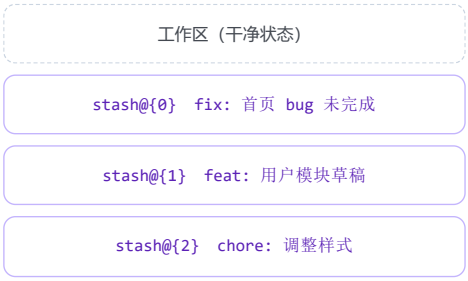
14 Stash

写到一半要切分支？先「存进抽屉」。

它解决什么问题

你正在 main 上改代码，改到一半还没法提交（代码是残缺的），但突然需要切到别的分支处理事情。 带着半成品切分支，很可能污染另一个分支。

Stash 就是把这些未提交的改动临时「打包收起来」，让工作区恢复干净，等回来再取出来。



Stash 是一个后进先出的栈，最新存入的在最上面（stash@{0}）

核心命令

```
# 存入抽屉（把当前所有未提交改动藏起来）
git stash
git stash push -m "首页样式改到一半"   # 带说明，推荐

# 包含未跟踪的新文件（默认不含）
git stash -u

# 查看抽屉里都有什么
git stash list

# 取出来并删除这条记录（最常用）
git stash pop

# 取出来但保留记录（想取多次时用）
git stash apply
git stash apply stash@{1}   # 取指定的某一条

# 查看某条 stash 具体改了什么
git stash show -p stash@{0}

# 删掉某条 / 清空全部
git stash drop stash@{0}
git stash clear
```

命令	取回改动	保留 stash 记录
`git stash pop`	✅	❌ 自动删除
`git stash apply`	✅	✅ 保留

⚠ **容易踩的坑**

① `git stash` **默认不包含未被 Git 跟踪的新文件**，新建的文件会被留在工作区。要用 `-u`；
② `pop` 时如果当前分支和存 stash 时的代码差异太大，可能产生冲突，需要手动解决；
③ Stash 是**本地的**，不会同步到远程，也不会备份。重要的东西还是老实提交到分支比较安全。

💡 **Stash vs Worktree 怎么选**

Stash：临时离开几分钟，改动还不完整，只想快速切走再回来。
Worktree：需要长时间并行在另一个分支工作，两个分支都要能随时编译运行。

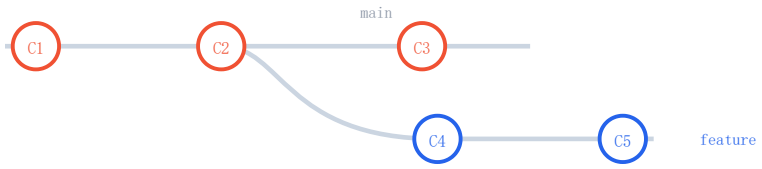
15 Rebase 变基

把「分叉的历史」重新捋成一条直线。

Rebase 做了什么

Merge 是「把两条线连起来」，会产生一个分叉的 merge commit。 Rebase 则是「把你的提交一个一个拔起来，重新插到目标分支的最新提交后面」，历史变成一条干净的直线。

① Rebase 之前（历史分叉）



② Rebase 之后（一条直线）



C4、C5 被「重新播放」到 C3 之后，hash 变了（带撇号），历史变成一条干净的直线

常用命令

```
# 把自己分支上的提交，重新接到 main 的最新提交之后
git switch feature
git rebase main

# 交互式 rebase：整理最近 3 次提交（合并、改信息、删除、排序）
git rebase -i HEAD~3

# pull 时用 rebase 模式，避免产生多余的 merge commit
git pull --rebase

# 出问题了，中止 rebase，回到操作前
git rebase --abort

# 冲突解决完，继续 rebase
git add <文件>
git rebase --continue
```

交互式 Rebase：整理提交历史的利器

执行 `git rebase -i HEAD~3` 后，编辑器会打开一个列表，每行前面有一个动作关键字：

关键字	含义
`pick`	保留这个提交（默认）
`reword`	保留提交但修改说明文字
`squash`	把这个提交合并进上一个提交
`fixup`	同 squash，但丢弃本条提交的说明
`drop`	删除这个提交
`edit`	暂停下来修改这个提交

Merge vs Rebase 对比

	Merge	Rebase
历史形态	保留分叉，有 merge commit	一条直线，无分叉
提交 hash	不变	**会改变**（生成新提交）
真实性	真实还原开发过程	历史被「美化」
适用	公共分支合并	整理自己的本地分支
风险	历史较乱	**改写历史，可能影响他人**

⛔ **Rebase 的黄金法则**

永远不要对「已经推送到远程、且有其他人在用」的分支执行 rebase。
因为 rebase 会重写提交 hash。你本地变基了，别人本地还是旧历史，一旦同步就会产生大量重复提交和诡异冲突，最后往往只能靠 `push -f` 强推来收拾，进而把队友的代码冲掉。

记住一句话：只在「属于你自己的、还没推送的」提交上做 rebase。

✅ **推荐的实践组合**

本地开发时：`git rebase -i` 整理提交 → 合并成一条干净的记录；
同步远程时：`git pull --rebase` 避免无意义的 merge commit；
合并到 main 时：走 Pull Request + Merge（保留分叉，保证安全可追溯）。

★ 命令速查表 全课汇总

把这张表打印出来贴在显示器旁，一周就能形成肌肉记忆。

环境与初始化

`git --version`	查看 Git 版本
`git config --global user.name "名字"`	配置用户名
`git config --global user.email "邮箱"`	配置邮箱
`git init`	初始化本地仓库
`git clone <url>`	克隆远程仓库到本地

日常提交

`git status`	查看当前状态（最常用）
`git add <文件> / git add .`	加入暂存区
`git commit -m "说明"`	提交
`git commit -am "说明"`	跳过 add 直接提交已跟踪文件
`git log --oneline --graph`	查看提交历史（带分支图）
`git diff / git diff --staged`	查看差异

分支

`git branch`	查看分支
`git switch -c <名字>`	创建并切换分支
`git switch <名字>`	切换分支
`git merge <分支>`	合并分支到当前分支
`git branch -d <分支>`	删除分支
`git worktree add ../路径 <分支>`	新建工作树

远程协作

`git remote add origin <url>`	关联远程仓库
`git remote -v`	查看远程地址
`git push -u origin main`	首次推送并绑定分支
`git push`	推送提交
`git pull`	拉取并合并（= fetch + merge）
`git pull --rebase`	拉取并变基（历史更干净）
`git fetch`	只下载不合并

撤销与回退

`git restore <文件>`	放弃工作区改动
`git restore --staged <文件>`	把文件移出暂存区

`git reset --soft HEAD~1`	撤销提交，改动回到暂存区
`git reset --mixed HEAD~1`	撤销提交和 add，改动回到工作区
`git reset --hard HEAD~1`	彻底回退（危险，公共分支禁用）
`git revert <commit-id>`	生成反向提交撤销某次改动（安全）

进阶

`git cherry-pick <commit-id>`	摘取单个提交到当前分支
`git stash / git stash pop`	临时存档 / 取出存档
`git stash list`	查看所有存档
`git rebase main`	把当前分支变基到 main
`git rebase -i HEAD~3`	交互式整理最近 3 次提交
`git rebase --abort`	中止变基
`git merge --abort`	中止合并

✅ **三句话总结整门课**

1. Git 用 **Commit** 记录快照，用**四区模型**（工作区 → 暂存区 → 本地仓库 → 远程仓库）管理流动。
2. **Branch** 让你并行开发，**Merge** 让成果汇合，**Conflict** 是人为裁决点。
3. 公共分支**永远不用 reset / rebase**，要撤销就用 **revert**——这条守住了，就基本不会出大事。